

PROJECT 1

FOUR DIGIT SEVEN SEGMENT DISPLAY

Project Description:

This project involves interfacing a 4-digit seven-segment display with an Arduino Uno to create a digital counter that increments from 0000 to 9999, updating every second. The display will be controlled using multiplexing, allowing efficient management of all four digits while using a minimal number of Arduino pins.

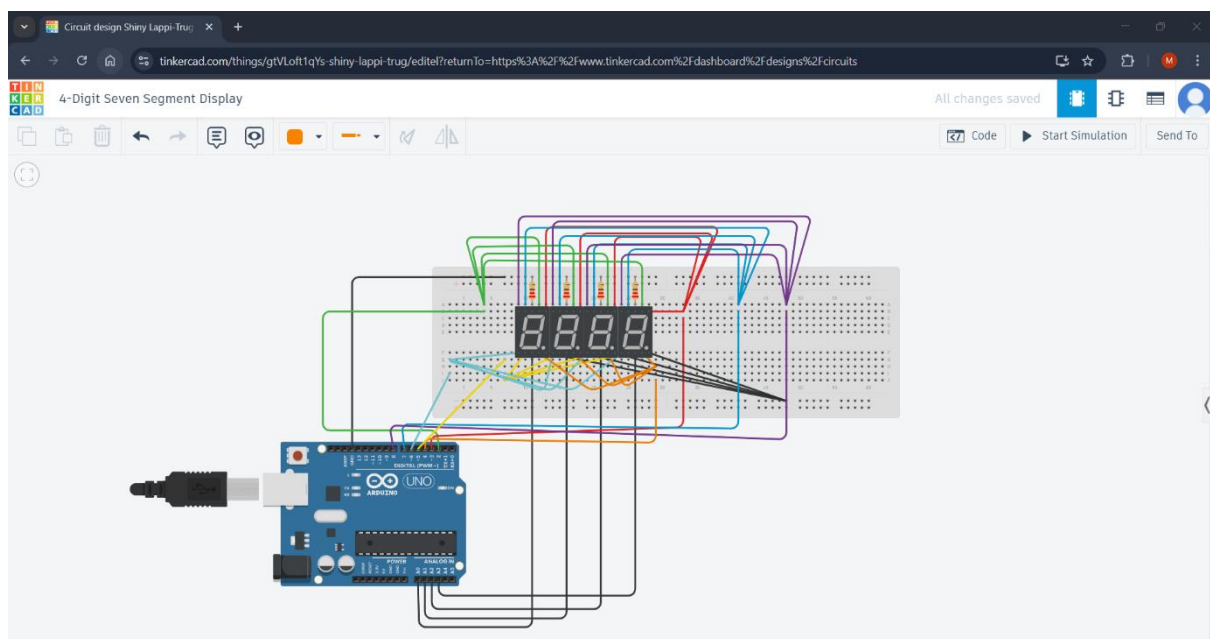
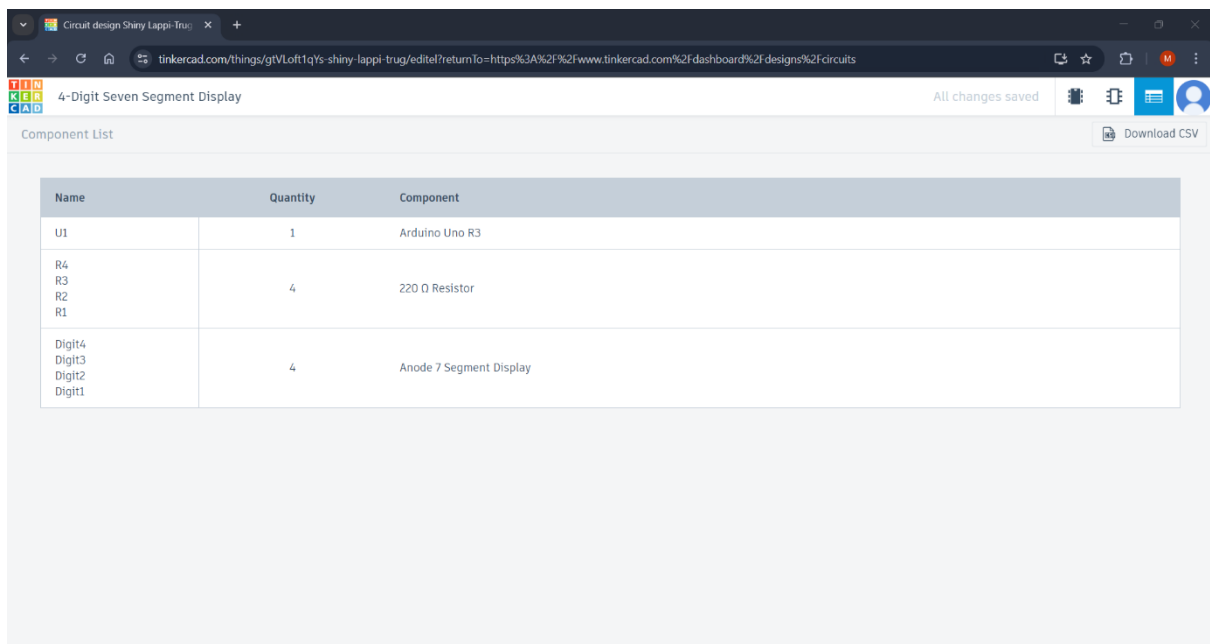


FIG: Design of Four Digit Seven Segment Display with Arduino UNO

Procedure:

1. Click on Create New Circuit.
2. Drag and drop components (ex: Arduino Uno R3, Seven-segment display, Resistors, Breadboard) from the component library onto the workspace.
3. Organize the components on the workspace for easy connections.
4. Use wires to connect components as per the circuit diagram. Click on a pin or terminal to start a wire. Drag the wire to the desired connection point and click to end it. Adjust the colour of wires to differentiate between connections.

5. Connect the segment pins of the seven-segment display to Arduino UNO R3 via current-limiting resistors (220Ω). Connect common pins to transistors or digit control pins for multiplexing.
6. Click on the code button in the top-right corner. Write or select block-based code to control the counter.
7. Write the Arduino code to increment the count every second, display it using multiplexing, and reset after 9999.
8. Click the Start Simulation button in the top-right corner to test your circuit.
9. Observe the seven-segment display updating the count from 0000 to 9999 in real time.
10. Verify the proper functioning of the circuit and make adjustments if necessary.



The screenshot shows the Tinkercad web interface. The browser tab is titled 'Circuit design Shiny Lappi-Trug'. The URL is 'tinkercad.com/things/gtVLoft1qYs-shiny-lappi-trug/edit?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fcircuits'. The project name is '4-Digit Seven Segment Display'. The component list table is as follows:

Name	Quantity	Component
U1	1	Arduino Uno R3
R4 R3 R2 R1	4	220 Ω Resistor
Digit4 Digit3 Digit2 Digit1	4	Anode 7 Segment Display

FIG: Components Required (Table)

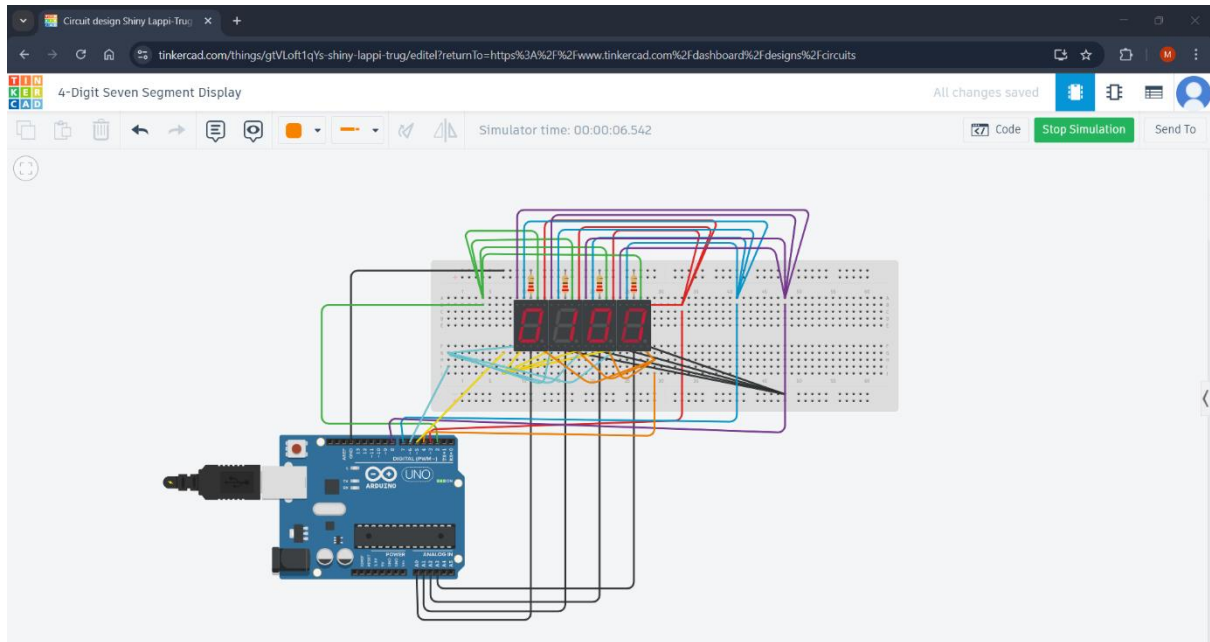


FIG: Simulation

Simulation Video:

<https://drive.google.com/file/d/1q9kUHVJHtnfXuqY6fCwOUf5KSMn6PHov/view?usp=drivesdk>

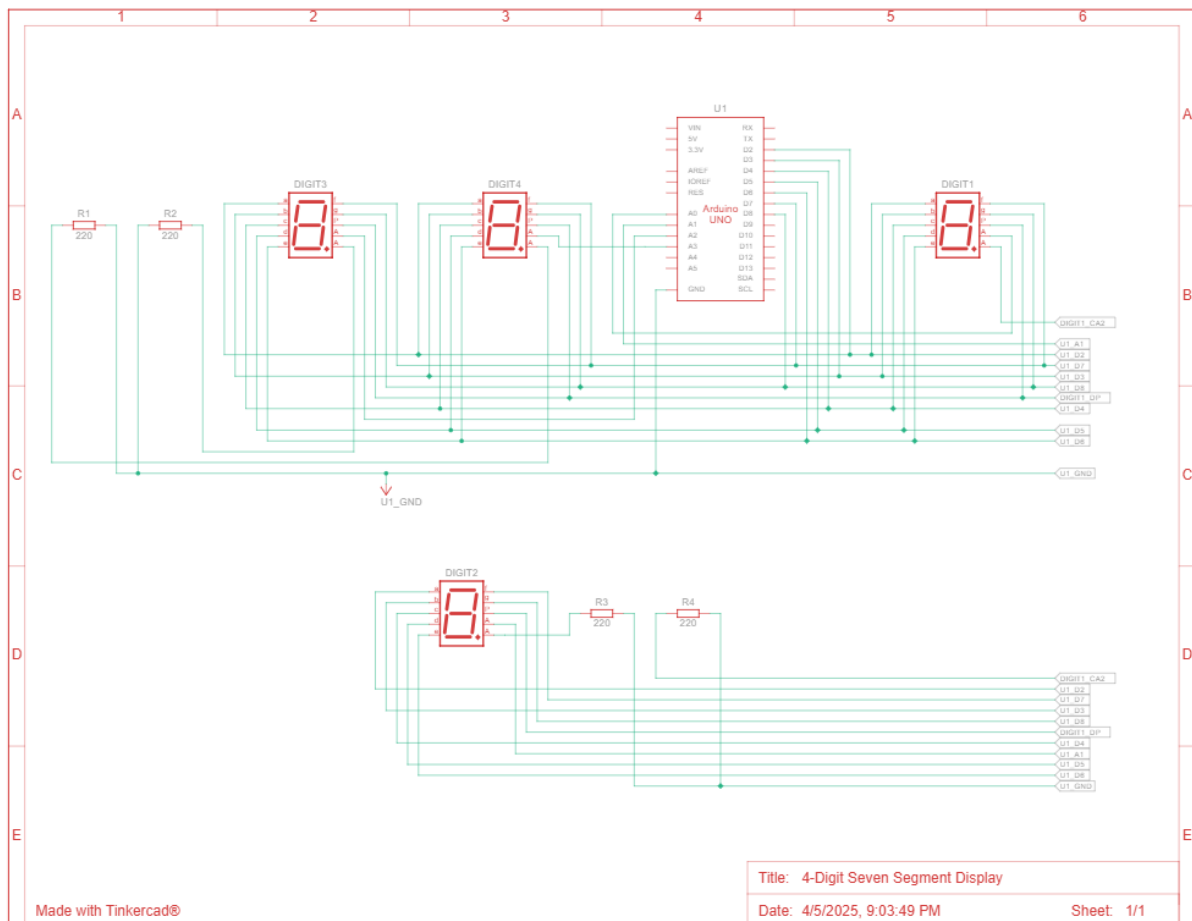


FIG: Schematic View

Code:

```
byte segValue[10][7] = {
  {0,0,0,0,0,0,1}, // 0
  {1,0,0,1,1,1,1}, // 1
  {0,0,1,0,0,1,0}, // 2
  {0,0,0,0,1,1,0}, // 3
  {1,0,0,1,1,0,0}, // 4
  {0,1,0,0,1,0,0}, // 5
  {0,1,0,0,0,0,0}, // 6
  {0,0,0,1,1,1,1}, // 7
  {0,0,0,0,0,0,0}, // 8
  {0,0,0,0,1,0,0}, // 9
};
```

```

byte segPin[8] = {2, 3, 4, 5, 6, 7, 8, 9}; // (a, b, c, d, e, f, g, dp)
byte digitPin[4] = {A0, A1, A2, A3}; // Control pins for each digit

void setup() {
    for (int i = 0; i < 8; i++) {
        pinMode(segPin[i], OUTPUT);
    }
    for (int i = 0; i < 4; i++) {
        pinMode(digitPin[i], OUTPUT);
        digitalWrite(digitPin[i], LOW);
    }
}

void loop() {
    for (int i = 0; i < 10000; i++) { // Display numbers from 0000 to 9999
        display_N(i);
        delay(10); // Adjust refresh rate
    }
}

void display_N(int num) {
    int units = num % 10; // Extract 1st digit (rightmost)
    int tens = (num / 10) % 10; // Extract 2nd digit
    int hundreds = (num / 100) % 10; // Extract 3rd digit
    int thousands = (num / 1000) % 10; // Extract 4th digit (leftmost)
    for (int i = 0; i < 10; i++) { // Refresh loop
        segOutput(3, units, 0);
        segOutput(2, tens, 0);
        segOutput(1, hundreds, 0);
        segOutput(0, thousands, 0);
    }
}

```

```
void segClear() {  
    for (int i = 0; i < 8; i++) {  
        digitalWrite(segPin[i], HIGH);  
    }  
}  
  
void segOutput(int d, int number, int dp) {  
    segClear();  
    digitalWrite(digitPin[d], HIGH);  
    for (int i = 0; i < 7; i++) {  
        digitalWrite(segPin[i], segValue[number][i]);  
    }  
    digitalWrite(segPin[7], dp); // Decimal point  
    delayMicroseconds(1000);  
    digitalWrite(digitPin[d], LOW);  
}
```

Simulation Link:

https://www.tinkercad.com/things/gtVLof1qYs-4-digit-seven-segment-display?sharecode=yntc__rN8juy7N93fwunfWVeFKNLCGUSuOp86I6SSpM

Conclusion:

This project successfully demonstrated the interfacing of a 4-digit seven-segment display with an Arduino to count from 0000 to 9999. The use of multiplexing allowed efficient control of all four digits, and the implementation of loops and delays ensured accurate and consistent updates every second. Future enhancements could include adding external inputs like push buttons or sensors to control the count dynamically, implementing serial communication for remote control and monitoring, and expanding the project to display custom messages or additional functionalities such as a countdown timer or stopwatch.

PROJECT 2

LED CONTROL USING SERIAL MONITOR

Project Description:

This project involves the implementation of an Arduino-based system to control an LED, monitor a switch's state, and display both states on a serial monitor. Additionally, the project allows for the control of LED blinking through serial commands.

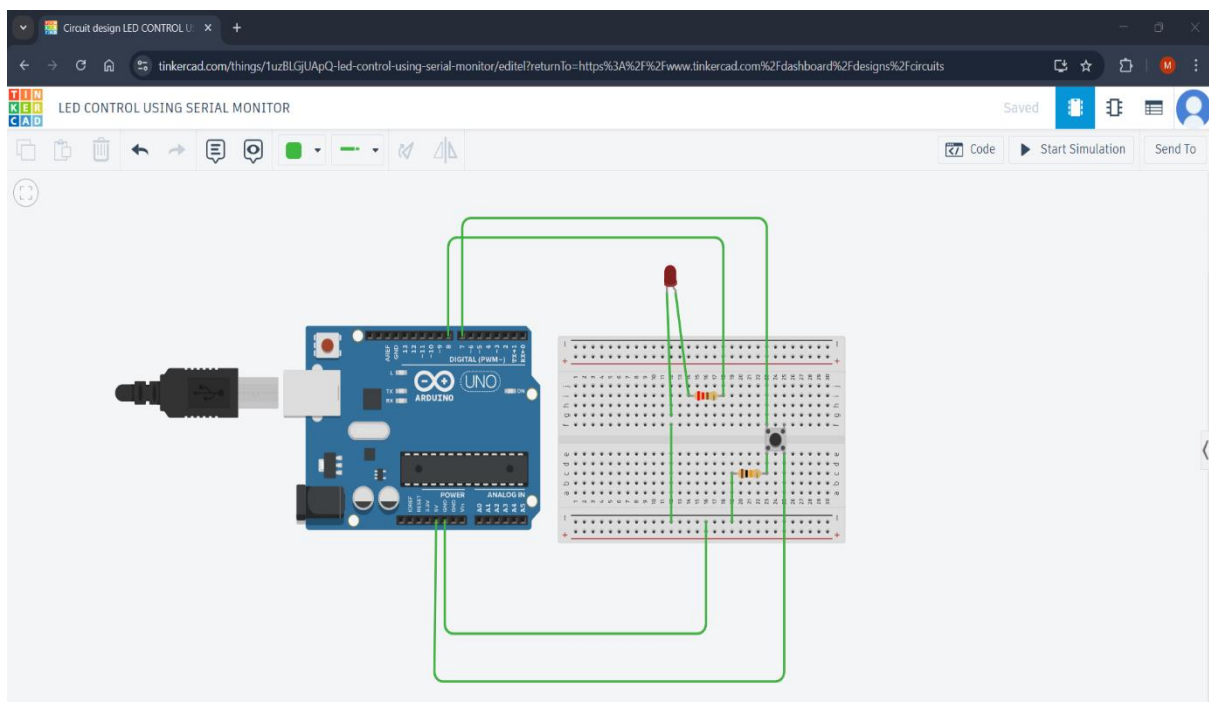


FIG: Design of LED control

Procedure:

1. Click on Create New Circuit.
2. Drag and drop components (ex: resistors, LED, switch, Arduino UNO R3) from the component library onto the workspace.
3. Organize the components on the workspace for easy connections.
4. Use wires to connect components as per the above figure. Click on a pin or terminal to start a wire. Drag the wire to the desired connection point and click to end it. Adjust the colour of wires to differentiate between connections.
5. Connect the pins on the Arduino to the respective components.

6. Click on the Code button in the top-right corner. Write or select block-based code to control your circuit.
7. Click the Start Simulation button in the top-right corner to test your circuit.
8. Open the Serial Monitor from the simulation controls.
9. Type 1 in the Serial Monitor to turn the LED ON.
10. Type 0 in the Serial Monitor to turn the LED OFF.
11. Verify the status of the LED on serial monitor.

Circuit design LED CONTROL U... x +

tinkercad.com/things/1uzBLGjUApQ-led-control-using-serial-monitor/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fcircuits

LED CONTROL USING SERIAL MONITOR Saved

Component List Download CSV

Name	Quantity	Component
U1	1	Arduino Uno R3
S1	1	Pushbutton
D1	1	Red LED
R1	1	220 Ω Resistor
R2	1	10 k Ω Resistor

FIG: Components Required (Table)

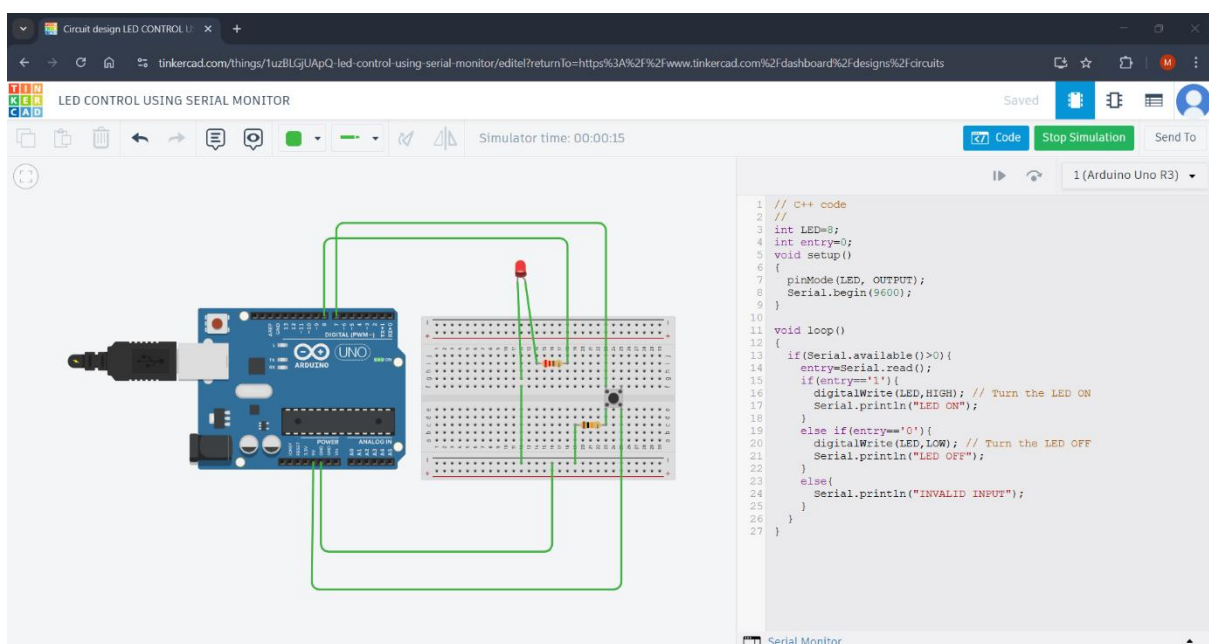


FIG: Simulation

Simulation Video:

https://drive.google.com/file/d/1psVigUWtypd_7NEZ4cwWUtDoiXfSxHuL/view?usp=drive_sdk

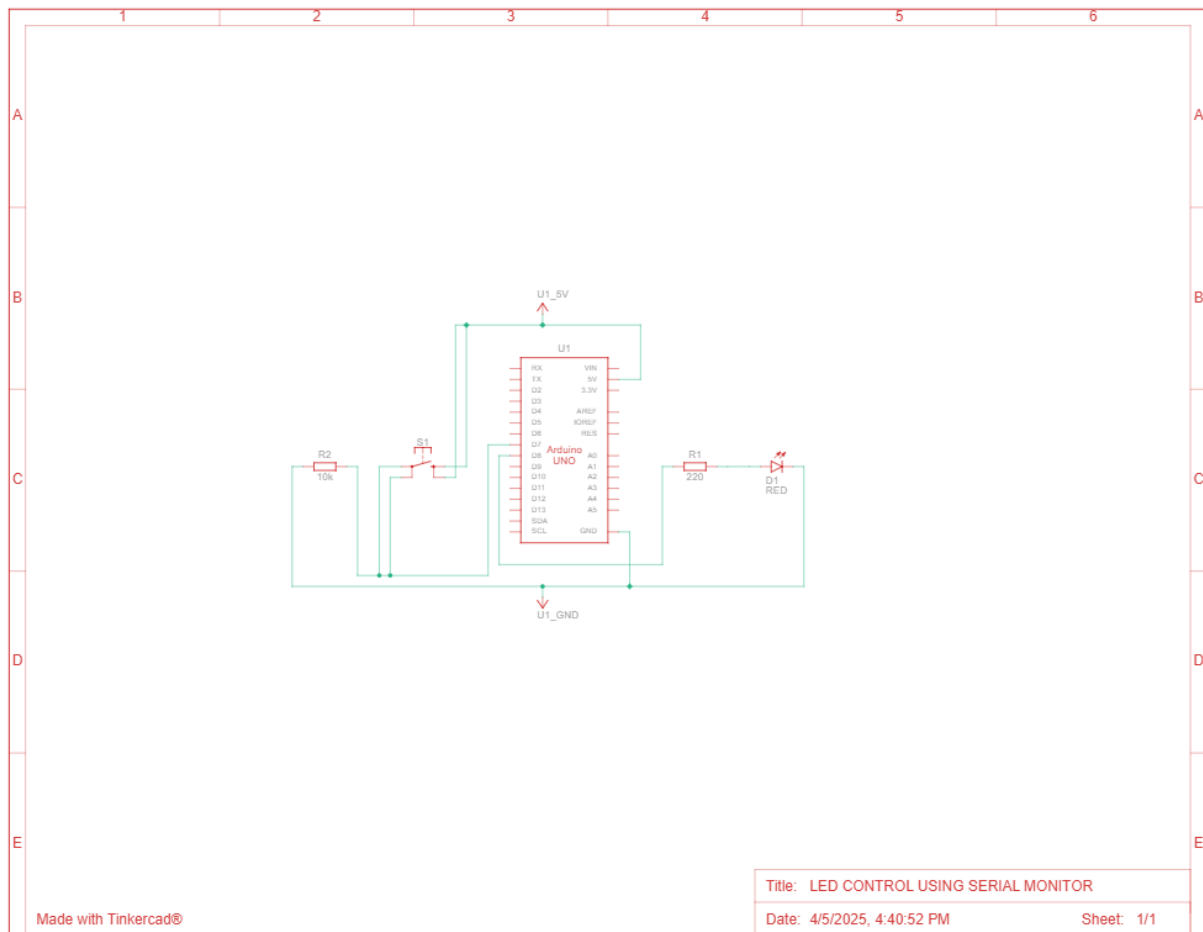


FIG: Schematic View

Code:

```
int LED=8;
int entry=0;
void setup()
{
  pinMode(LED, OUTPUT);
```

```
Serial.begin(9600);  
}  
void loop()  
{  
  if(Serial.available()>0){  
    entry=Serial.read();  
    if(entry=='1'){  
      digitalWrite(LED,HIGH); // Turn the LED ON  
      Serial.println("LED ON");  
    }  
    else if(entry=='0'){  
      digitalWrite(LED,LOW); // Turn the LED OFF  
      Serial.println("LED OFF");  
    }  
    else{  
      Serial.println("INVALID INPUT");  
    }  
  }  
}
```

Simulation link:

https://www.tinkercad.com/things/1uzBLGjUApQ-led-control-using-serial-monitor?sharecode=6uS7lDJ8ZiqjsiBtcWnx3mdl_GvN9OctoK4r_fdpVuo

Conclusion:

This experiment successfully demonstrated how to interface a switch and an LED with an Arduino. We were able to read the switch status and display it on the Serial Monitor, while also controlling the LED using serial commands. The project helped in understanding how digital inputs and outputs work, how to use the Serial Monitor for communication, and how to apply conditional logic in real time.

PROJECT BY:

Mallampally. Jayantha Siva Srinivas